Session_10 activities worksheet

Online handouts: listings of Circle class files, private_vs_public.cpp, and square_test.cpp; "Using the GDB Debugger" and GDB summary sheet.

In this session we'll again take a brief break from physics and computational methods to look at some programming tools and take a further look at C++ classes.

Your goals for this worksheet:

- Do a bit more with C++ classes.
- Try out the GDB debugger and consider optimization and profiling.
- Try out rsync.

A) Revisiting area.cpp with a C++ Class

Our first simple C++ code from Session_01 used procedural programming, with the focus on the formula (which would typically be coded as a function) for calculating the area. Now we revisit it from an object oriented perspective.

1. Look at the online printout with test_Circle.cpp and the old area.cpp, with the Circle class definitions. Note how all of the details of the area calculation are now hidden. *Any questions on the Class definition? Predict where in the test_Circle.cpp code the two circles will be created and where they will be destroyed. Why do we define get_radius and set_radius methods?*

   → No questions about the class definition. The circles will be created with the constructor like in Circle my_first_circle(my_radius). The first circle will be destroyed with the destructor, likely at the end of the program since no explicit destruction call is in the code. The second circle will be destroyed once we exit its scope (outside the curly braces). We define the get_radius and set_radius methods in the header Circle.h and implement them in Circle.cpp.

2. Using make_test_Circle, compile and link test_Circle.cpp with the class files. Run it and note where the circles are created and destroyed. *Do you understand why they are destroyed in this order?*

   → Yes, I understand why they are destroyed in this order.

3. Add a method to Circle.cpp (and Circle.h!) so that we can get the circumference from my_circle.circumference(). Try it out in test_Circle.cpp. *Did you succeed?*

   → Yes, it worked.

4. Take a look at the private_vs_public.cpp code, which is a self-contained class and main program. Notice how the main program can access and change the x value and use the xsq function. But try changing from x to y in the statements getting, printing, and changing the value of x. *What happens? Why does the get_y function work?*

   → You run into errors and issues because the variable and function y and ysq are private and cannot be accessed like x and xsq. The get_y function works because it's public, meaning it can be accessed outside the class itself.

5. (Extra, only do this if you are fast.) Make a Sphere class with Sphere.cpp and Sphere.h, and try it out by adding to test_Circle.cpp to create a Sphere and print out its radius. *Did you succeed?*

   → Yes, it worked.

B) Using the GDB Debugger

We'll step through a contrived example that illustrates the basic commands and capabilities of a debugger (in this case, the GNU debugger gdb). We will use the command-line ("no windows") version of gdb. There are graphical interfaces (such as ddd) that are much nicer to use for more extensive debugging, but it will be worthwhile to start with the simple, primitive version.

1. If you are a Mac user, you may be able to install gdb with `brew install gdb` (let me know if that works). Otherwise, you really need to use OSC for this one.
2. When debugging, you may find it convenient to have two terminal windows open: one to re-compile and link a sample code and another to run gdb in. (Actually, you can interact with gdb directly through emacs, but we won't go into that here.)
3. Go through the example from the (printed!) handout "Using the GDB Debugger". The code to debug is check_primes.cpp (a copy is also provided, called check_primes_orig.cpp, so that you can go back to the original if necessary). We've also created a makefile that doesn't use any of our usual warning flags (at first!). *Did you succeed?*

   → Yes, it worked.

C) Optimization 101: Squaring a Number

One of the most common floating-point operations is to square a number. Two ways to square x are: pow(x,2) and x*x. Let's test how efficient they are.

1. Look at the printout for the square_test.cpp code. It implements these two ways of squaring a number. The "clock" function from time.h is used to find the elapsed time. Each operation is executed a large number of times (determined by "repeat") so that we get a reasonably accurate timing.

2. We've set the optimization to its lowest value, -O0 ("minus oh zero"), to start in make_square_test.
3. Compile square_test.cpp (using make_square_test) and run it. Adjust "repeat" if the minimum time is too small. *Record the times here. Which way to square x is more efficient?*

   → Using my system, using pow took 0.627914s and using x*x took 0.0826s. Squaring x manually (not using pow) is more efficient.

4. If you have an expression (rather than just x) to square, coding (expression)*(expression) is awkward and hard to read. Wouldn't it be better to call a function (e.g., squareit(expression)? Add to square_test.cpp a function:
   double squareit (double x)
   that returns x*x. Add a section to the code that times how long this takes (just copy one of the other timing sections and edit it appropriately, making sure to keep the "final y" cout statement). *How does it compare to the others? What is the "overhead" in calling a function (that is, how much extra time does it take)? When is the overhead worthwhile?*

   → Using squareit is the slowest method at 0.107438s. The overhead compared to doing x*x is about 0.025s. The overhead is worthwhile based on complexity and maintainability. Squaring x is simple for this case, but if you start using a more complicated formula that you're going to type twice to square it, it's a recipe for disaster. It's also more maintainable if we need to change the logic or add error checking because the function is in one place, instead of being strewn all over the place in multiple expressions.

5. Another alternative, common from C programming: use #define to define a macro that squares a number. Add
   #define sqr(z) ((z)*(z))
   somewhere before the start of main. (The extra ()'s are safeguards against unexpected behavior; **always** include them!) Add a section to the code to time how long this macro takes; what do you find?

   → It's actually the fastest method, taking 0.082314s, and is very comparable to x*x.

6. One final alternative: add an "inline" function called square:
   inline double square (double x) { return (x*x); };
   that is a function prototype **and** the function itself. Put it up top with the squareit prototype. Add a section to the code to time how long this function takes. *What is your conclusion about which of these methods to use?*

   → The inline method is comparable to using the function squareit at 0.112627s. From this, for our purposes in this code, the best method to use would either be using the C-style defined sqr function or just simply hard-coding x*x. However, for better maintainability and generalization,

I would personally use sqr, since you can input more complicated expression which could become erroneous by doing x*x.

7. While defines have their charm, there are a few odd things about them. Make a copy of your program and change your program such that instead of squaring i it squares the sine of i. Also, instead of assigning the sine of i to x and then squaring it, let's write it as a single line everywhere (e.g., write sqr(sin(double(i))) for the version with the define). *Compare again the time it takes for the different approaches. What differences do you note?*

   →I notice higher times across the board (which makes sense due to using sin(i) instead of just i). In terms of speed, it seems that sqr is still the fastest method, but it's still very close to x*x. I also notice that its relationship to something like squareit is not very far off in terms of total elapsed time. The general order seems to be relatively the same.

8. One final warning about defines. In C++ you can increase an integer variable mid-expression using the ++ operator. Add the following code somewhere toward the end of your program:

```
int n = 5;
int m = 5;
cout << "The square of " << n << " is ";
cout << sqr(double(n++));
cout << " and n is now " << n << endl;
cout << "The square of " << m << " is ";
cout << square(double(m++));
cout << " and m is now " << m << endl;
```

   Also, please remove the -Werror in the makefile to prevent the compiler from terminating due to the well-meaning warning this code might generate.

   Before you run it take a guess as to what the output is going to be. Were you correct? What do you conclude from the result about using defines?

   →From the code, sqr(double(n++)) will be seen by the code as (double(n++)*double(n++)), which will evaluate to 5*6=30, and then n will be 7 at the end. For m, the square function will see double(m)*double(m) which will yield 25 and then be incremented to m=6. This is the result I got back. From this result, I conclude that macro defines can be dangerous because they are text-replacements that may not necessarily understand C++ logic. They can be powerful and fast, but they inherently lack scope and can be trapped by substitutions like this n++ issue.

9. Finally, we'll try the simplest way to optimize a code: let the compiler do it for you! Go back to the copy you made in step 7 (but please do not overwrite the version with the sines but leave a copy of it somewhere for us to grade). Change the compile flag -O0 (no optimization) to -O2 (that's the uppercase letter O, not a zero). Recompile and run the code. *How do the times for each operation compare to the times before you optimized? What do you conclude?*

→The times for each operation are significantly faster. On top of that, they are all feasible solutions. Using x*x and sqr(x) were still the fastest, but squareit and square were comparible with the same order of magnitude of 1e-6s. The pow function was still the slowerst, but still fast at 2.7e-5s. From this, you could also conclude that for simple cases like squaring, once optimized any of the options can be used quickly (but maybe not safely, as the optimizations may edit output results/accuracy).

10. In your project programs, once they are debugged and running, you'll want to use the -O2 (or maybe -O3) optimization flag.

D) Profiling

A "profiling" tool, such as gprof, allows you to analyze how the execution time of your program is divided among various function calls. This information identifies the candidate sections of code for optimization. (You don't want to waste time optimizing a part of the code that is only active for 1% of the run time!) We'll use the eigen_basis_class.cpp code from an earlier Session as a guinea pig. [Note: gprof is not supported on Mac OS X. For this Session, everyone should run on Linux again (either directly or by ssh as described above).]

1. To use gprof, compile and link the relevant codes with the -pg option. You can do this most easily by editing make_eigen_basis_class and adding -pg to **BOTH** the CFLAGS and LDFLAGS lines. (Note that make_eigen_basis_class has $(MAKEFILE) added to two lines in section 4 to ensure that the codes are recompiled if the makefile changes.)
2. Execute the program as usual (choose a fairly large basis size so that it takes a while to execute, building up statistics), which generates a file called gmon.out that is used by gprof. The program has to exit normally (e.g., you can't stop it with control-c) and any existing gmon.out file will be overwritten.
3. Run gprof and save the output to a file (e.g., gprof.out):
   `gprof eigen_basis_class.x > gprof.out`
   Edit gprof.out and try to figure out from the "Flat profile" and the explanations where (i.e., in what functions) the program spends the most time. *What are the most time-consuming functions? Would you try to speed up the part that finds the eigenvalues?*

   → It seems the most time-consuming function is ho_radial, having the most calls and taking the most ns/call. No, I wouldn't try to speed up the part that finds the eigenvalues, as it doesn't take a lot of calls, nor does it take a lot of time, relatively speaking.

4. Now change the makefile so that -O3 optimization is used. Run gprof and save the output to a new file:
   `gprof eigen_basis_class > gprof2.out`
   Compare gprof.out and gprof2.out. *What has* -O3 *done for you?*

→ The flag -O3 has increased the speed of the program, and reduced the ns/call for ho_radial, but it is still the most significant portion of the program runtime.

E) Introduction to Rsync

Rsync is a backup or mirroring tool that only copies the *differences* of files that have changed. It can do this transfer in compressed form and using ssh for security (both recommended!). So updates are fast, efficient, and secure. Here we'll make a trial run so that you get the basic idea.

1. [The following is from Session_05.] **Bash (tcsh) aliases.** The .bashrc (.cshrc.more) file sits in your home directory and is "sourced" when you start an interactive terminal. Take a look at the sample .bashrc (.cshrc.more) file printout (these files, without the ".", are in the Session_10 zip file). Copy any aliases of interest to your own .bashrc (.cshrc.more) or the whole thing using cp bashrc ~/.bashrc (cp cshrc.more ~/.cshrc.more) if you don't have one already. Activate the changes with the command: source ~/.bashrc (source ~/.cshrc).
2. Take a look in the bashrc or cshrc.more file for the definitions "rsyncbackup" and "rsyncmirror". These aliases use some of the common rsync options. The difference is that rsyncmirror deletes files at the destination that don't exist at the source; be careful of this!
3. Create a directory in your home directory called "5810_backup" to use for testing rsync (i.e., "`mkdir 5810_backup`").
4. Our session is on the candinal cluster of OSC. To see how rsync works, we will transfer the files to the other cluster, pizter. They share the home directories, so all you will notice, if everything goes right, is that the files were copied from one directory in your account to another, but they were really copied from cardinal to pitzer! Go to the parent directory of session_10. Give this command:

```
rsyncbackup session_10 pitzer.osc.edu:5810_backup
```

If you are asked if you want to continue connecting, answer yes. You'll probably be asked for your (OSC) password as well. You should see a list of files transferred and some statistics. If you have access to another computer, you can replace pitzer.osc.edu by that computers name (and possibly put "your_user_name@" in front), but make sure to make that 5810_backup directory on the destination machine first.

5. Check that the directory was transferred. Now go back to the original session_10 directory and change one file. (E.g., edit one of the .cpp files and add a comment.) Then repeat the rsync transfer (from the appropriate directory), using "!rsync" to avoid typing the entire alias. You should find that only the changed file is updated. *Did you succeed? If not, what went wrong?*

   → Yes, it worked.

F) Reflection

Write down which of the six learning goals on the syllabus this worksheet addressed for you and how:

→ This session helped me learn some more tools of computational physics, especially with debugging and other computing tools like GDB, RSync, and GProf. I also learned more about the limitations of computational solutions, specifically in the realm of optimization (which methods of calculating things are the fastest, what causes memory issues, etc.).